

Article

Benchmarking YOLOv8 Variants for Object Detection Efficiency on Jetson Orin NX for Edge Computing Applications

Hadeel Muhammad Aljami ¹, Nouf Abdullah Alrowais ¹, Anfal Mohsen AlAwajy ¹, Shog Osama Alhrgan ¹,
Raghad Abdullah Aldwaani ¹, Motasem Samer Alsawadi ^{1,*}, Najam Us Saqib ², Syed Salman Alam ²,
and Reem Alsubaie ²

¹ King Abdulaziz City for Science and Technology (KACST), Riyadh 11442, Saudi Arabia; haljami@kacst.gov.sa (H.M.A.); nalrowais@kacst.gov.sa (N.A.A.); aalawajy@kacst.gov.sa (A.M.A.); shoug.alhr@gmail.com (S.O.A.); raghadaldawani807@gmail.com (R.A.A.)

² Elm, Riyadh 12382, Saudi Arabia; nsaqib@elm.sa (N.U.S.); syalam@elm.sa (S.S.A.); reealsubaie@elm.sa (R.A.)

* Correspondence: malswadi@kacst.gov.sa

Abstract

Edge AI is redefining the deployment of computer vision systems by enabling real-time inference directly on resource-constrained edge devices. This shift offers significant advantages in terms of reduced latency, data privacy, and operational autonomy in bandwidth-limited computing environments. In this paper, we present a systematic performance benchmarking of multiple variants of YOLOv8 on the NVIDIA Jetson Orin NX platform, focusing on object detection tasks. We evaluate inference latency, frame throughput, and computational resource usage across varying input sizes and model complexities. Furthermore, we validate the deployment effectiveness through practical use cases, such as vehicle and package detection. Our findings show that the TensorRT model outperforms PyTorch by 17.7% at a batch size of 2, although PyTorch presents greater stability at larger batch sizes (e.g., 8), where TensorRT encounters resource constraints. In terms of memory usage, it increases linearly with batch size: 69% from batch 1 to 4, with TensorRT requiring 429.20 MB at batch size 2 compared to PyTorch's 451.24 MB. Furthermore, the processing time per image decreases by 42% when scaling from batch size 1 to 4, highlighting a critical saturation point for edge resources. In summary, the results provide insight into the trade-offs between model size and speed, offering guidance for selecting detection architectures tailored to real-time edge applications.

Keywords: edge AI; YOLOv8; object detection; performance benchmarking; NVIDIA Jetson Orin NX; real-time applications; model optimization



Academic Editors: Paolo Bellavista and Riccardo Venanzi

Received: 7 December 2025

Revised: 8 January 2026

Accepted: 9 January 2026

Published: 1 February 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

Edge AI marks a significant shift in how computer vision applications are designed and executed. This transformation allows data to be processed intelligently on local devices such as smartphones, embedded processors, and dedicated edge hardware—rather than relying on remote, cloud-based servers. As a result, the need to transmit high-volume visual data to cloud-based infrastructure is significantly reduced [1]. The integration of artificial intelligence with edge computing has led to ongoing improvements in hardware efficiency, neural network design, and model optimization. These advancements enable the execution of complex computer vision tasks on devices with limited processing capacity [2].

Deployment of computer vision-based deep learning models in edge applications requires careful consideration of multiple interdependent trade-offs among inference accuracy,

computational load, and energy consumption. From a computational standpoint, balancing model complexity and inference speed is challenging. For instance, while YOLOv8-medium offers higher accuracy, it imposes greater processing and resource demands compared to streamlined architectures such as SSD MobileNet [3]. Moreover, energy constraints are a critical factor; edge devices usually operate under strict power budgets, necessitating careful optimization of model architectures and processing pipelines [4]. Additionally, the vast array of architecture choices complicates deployment. Developers must navigate options ranging from model compression techniques, such as quantization and pruning, to hardware-centric optimizations provided by frameworks like TensorRT and OpenVINO [5]. These trade-offs are essential for achieving acceptable performance within the thermal, memory, and processing constraints of the destination edge hardware [6,7].

In this context, performance encompasses several facets, including inference latency (ms per frame), throughput (frames per second), resource efficiency (GPU/CPU load and memory consumption), power/energy efficiency, and detection efficiency (mean average precision, mAP) [8–10]. Defining these dimensions is essential to position benchmarking results in relation to prior work. Representative edge benchmarking efforts include broad comparative evaluations on embedded devices and quantitative studies of object detectors on edge platforms [3,8,10]. Recent work on deploying large language and large AI models at the edge reports similar memory and latency constraints, both in IoT-focused LLM platforms [11] and in pruning-aware co-inference of large models over wireless edge networks [12]; while the present study is limited to CNN-based YOLOv8 object detectors, the benchmarking approach and resource usage analysis can be extended to transformer-based models on Jetson-class or similar edge devices.

The growth of edge computer vision in the last few years has been significant, spanning autonomous vehicles, smart surveillance, agricultural monitoring, industrial quality inspection, and medical diagnostics [13]. These applications use deep learning models to perform tasks such as object detection, image classification, semantic segmentation, and pose estimation directly on edge devices. Integrating these capabilities at the edge facilitates real-time visual analysis, which is necessary to support applications that demand rapid decision-making [14].

Data processing on the edge offers several significant benefits compared to cloud-based approaches. One of the primary advantages is significantly reduced latency, as data does not need to be transmitted to remote servers for processing and then returned for execution [15]. This reduction in processing time is crucial for real-time applications such as autonomous navigation, where even milliseconds of delay can negatively impact decision-making [16]. Research has demonstrated that edge-based computations can achieve an end-to-end latency reduction as high as 85% compared to cloud-based deployments, especially in bandwidth-constrained environments [17]. In time-critical scenarios—such as collision avoidance or emergency response systems—this latency reduction can be the difference between success and failure [18].

Edge computing also significantly improves security and privacy by confining sensitive visual data to the local environment. This eliminates the need to transmit data over potentially insecure networks, thereby reducing the risk of interception or unauthorized access [19]. The privacy-preserving aspect of edge computing has gained importance due to increasing public awareness and evolving data protection regulations [20]. Edge processing supports a privacy-by-design approach, wherein only processed outputs or anonymized data are transmitted when necessary, while raw visual data remains locally protected [21]. For applications such as surveillance, facial recognition, or medical imaging, local processing ensures that sensitive data remains secure without compromising system functionality [22].

The YOLO (You Only Look Once) family of object detection models has become a cornerstone in real-time computer vision on edge devices. Since the introduction of the original YOLO architecture in 2016 [23], these single-stage detectors have continually improved the speed–accuracy trade-off by viewing object detection as a regression task rather than a classification task. Through progressive improvements, the YOLO architecture has evolved to incorporate novel concepts such as feature pyramid networks, anchor-free detection schemes, and attention mechanisms. This evolution culminated in the YOLOv8 series, which offers various model sizes optimized for distinct deployment scenarios [24]. The fundamental approach of processing the whole image in a single pass makes YOLO highly suitable for edge devices, where fast inference is imperative [25].

NVIDIA's Jetson Orin NX is widely used for edge AI in robotics, autonomous systems, and embedded vision. This is due to its favorable combination of energy efficiency (10–25 W) and high on-device compute capability (up to 100 TOPS). Consequently, it provides a representative and practically relevant benchmark platform for contemporary object detection workloads [3,26]. This choice is further motivated by its adoption in autonomous/robotic research and the maturity of the NVIDIA TensorRT toolchain on Jetson-class devices [27,28].

While YOLOv8 has been adopted in various edge computing scenarios, a notable gap remains in the system-level characterization of its behavior under hardware-accelerated deployment on unified-memory platforms like the Jetson Orin NX. Existing benchmarks often emphasize peak indicators, such as maximum FPS or minimum latency, under default or low-batch settings. However, these studies frequently overlook the nonlinear interactions among batch size, memory utilization, and inference delay. Furthermore, they rarely report the critical points at which inference engines become unstable or fail. This limitation is critical for practitioners attempting to maximize throughput via aggressive batching on resource-constrained devices. Accordingly, the present work goes beyond reporting raw performance numbers to quantify the operational boundaries of YOLOv8 variants under both PyTorch and TensorRT deployments. We highlight where TensorRT delivers substantial speed-ups and where it encounters resource-exhaustion issues while PyTorch remains stable.

In light of these opportunities and challenges, this paper benchmarks YOLOv8 variants across key performance dimensions (latency, FPS, memory) to provide practical guidelines for resource-aware edge deployments. In particular, our primary contributions include the following:

- We identify the critical stability limits of Jetson Orin NX under high-throughput conditions, precisely mapping TensorRT failure points at high batch sizes for YOLOv8 variants.
- We contrast cross-format deployment (PyTorch vs. TensorRT) and demonstrate the substantial efficiency gains enabled by hardware-specific optimizations on Jetson-class edge processors, quantifying performance differences of up to 17.7%.
- We investigate resolution scaling and batch size optimization effects on performance–memory trade-offs, documenting how GPU memory scales linearly while performance benefits show diminishing returns.
- We present two real-world use cases—vehicle and package detection, and counting—validating our experimental findings in practical deployment scenarios.
- We provide key guidelines for practitioners to select the most appropriate YOLOv8 variant depending on application constraints (e.g., Nano for lightweight real-time systems, X for accuracy-critical tasks), thereby providing practical insights for edge model selection.

2. Related Work

2.1. Edge AI Deployment

Edge AI is defined as the deployment of machine learning models on edge devices, enabling real-time processing near data sources; it reduces latency, improves privacy, and enhances effectiveness by reducing dependency on cloud processing [29]. The use of edge AI can be found in multiple domains with the deployment of IoT functionality and advanced machine learning frameworks.

In industrial domains, edge AI enhances operational workflows by optimizing resources and performs sophisticated tasks such as object detection, improving operational efficiency and minimizing errors [30]. Real-time data analysis from sensors enables predictive maintenance, reducing downtime on equipment [31]. Healthcare applications involve patient monitoring with IoT networks, providing real-time response in emergency cases, while also enhancing data privacy and security [32,33].

In smart cities, edge AI plays a growing role in real-time traffic management and environmental monitoring, thereby providing timely insights for urban planning [31,34]. Autonomous vehicles utilize edge AI for navigation, obstacle avoidance, and hazard detection, enhancing safety and response in driving systems [31,35]. In the government and public sectors, edge AI technologies are used in various applications, including facial recognition and social network analysis [36], with the agricultural industry benefiting through precision agriculture applications, while the finance sector utilizes edge AI to detect fraud [30].

Despite its advantages, edge AI is plagued by various challenges, such as limited computational resources, higher energy requirements, and vulnerability to security attacks. Recent studies highlight enhancements in hardware, model simplification, and privacy-preserving algorithm application [35,37]. Model compression techniques such as pruning, quantization, and knowledge distillation are seen as efficient methods to deploy complex AI models on resource-limited devices [38]. Edge–cloud hybrid architectures allow for balancing computational loads. Specialized AI hardware such as GPUs, FPGAs, and ASICs fosters efficient model execution [39]. The incorporation of 5G technology is also poised to further strengthen abilities and system reliability [35].

Performance metrics employed in earlier edge computer vision research typically compare across a common set of dimensions: latency (ms/frame), throughput (FPS), and accuracy measures such as COCO mAP@0.5:0.95/mAP@0.5 [8–10,40]. At the system level, one also encounters instantaneous power consumption (W), energy per frame (J/frame), and normalized efficiency terms such as FPS/W for cross-device comparisons [6,7,10]. Resource footprints such as FLOPs, runtime, and memory are also reported to characterize scalability and embedded deployability [21,28,40]. Some works also use aggregate objectives such as energy–delay product to summarize accuracy–efficiency trade-offs [6,7]. Bounding these benchmarks also highlights that comparison results necessitate multi-dimensional reporting, keeping the current work in line with established practice in evaluation [8,10].

2.2. YOLO Model Benchmarking

We now provide a background on YOLO evolution (v5–v8) and edge relevance. YOLO variants have made architectural changes to improve accuracy–efficiency trade-offs for in-place deployment in resource-constrained situations since 2021 [3,8,10,40]. YOLOv5u added an anchor-free split head in the v5 family [41]; YOLOv6 implemented EfficientRep backbones, RepPAN aggregation, and a quantization-friendly decoupled head [42]; YOLOv7 added stretched ELAN scaling in combination with re-parameterized convolutional blocks [43]; and YOLOv8 formalized an anchor-free decoupled head in combination with a streamlined C2f backbone/neck and strong exportability in PyTorch, ONNX, and

TensorRT [40,44]. These optimizations minimize post-processing overheads and enable embedded GPU/TPU deployment, driving variant-level benchmarking to examine trade-offs across model size regimes [3,8,10,26]. These architectural refinements were explicitly designed to reduce compute overhead, enable precision-friendly deployment (FP16/INT8), and make YOLO variants increasingly suitable for real-time edge benchmarks.

Recent evaluations of YOLO models on edge devices reveal key findings into practical applications for resource-constrained environments. NVIDIA Jetson platforms have emerged as commonly used platforms, with Elhanashi et al. [45] indicating that YOLOv8n performs best in terms of FPS across the Jetson Nano, Xavier AGX, and Orin platforms. The Jetson Nano typically reaches 15 FPS with YOLOv4-tiny and is further improved with TensorRT optimizations [26]. Cost-effective alternatives such as Jetson Nano 2 GB achieve 12.8 FPS with the lightweight YOLO-RTUAV (5.5 MB) [46].

Alternative platforms such as Raspberry Pi with Intel NCS2 show 7.6× higher FPS using YOLOv3-tiny versus YOLOv3 [26]. In autonomous vehicle contexts, Ahmed et al. [47] reported YOLOv8 achieving 0.79 precision and a 0.69 F1-score on edge devices. Specialized variants address deployment challenges: Edge-YOLO reduces computational intensity by 70.3% and model size by 71.6% [48], while Squeezed Edge YOLO demonstrates 8× size reduction and 76% enhanced power efficiency [49].

YOLOv8 has shown success in numerous real-time applications and reached 97.6% mAP at IoU 0.5 in agricultural scenarios [50], as well as showing strong performance in healthcare applications, where YOLOv8n provides the perfect balance between accuracy and inference speed for stroke detection [45]. The YOLOBench platform [8] evaluates over 550 models by YOLO on various hardware platforms and shows that legacy models can still compete when coupled with modern detection approaches.

Despite these advances, challenges persist in balancing accuracy and efficiency. Raspberry Pi struggles with real-time processing despite its low power consumption [51], highlighting the complexity of cross-platform adaptation. The fundamental accuracy–efficiency trade-off requires thoughtful model choice based on use-case requirements and available hardware resources.

2.3. Comparative Benchmarks and Synthesis

To place our NVIDIA Jetson Orin NX research into context compared with prior edge deployments, Table 1 encapsulates four representative papers that report quantitative results (latency/FPS and/or mAP) for Jetson-class and corresponding edge platforms. The selection spans device generations and detector families as well as highlighting accuracy–efficiency trade-offs that provide the impetus for our interest in analyzing TensorRT and PyTorch formats for YOLOv8 across multiple batch configurations and different input resolutions.

As summarized in Table 1, YOLOv8n on Orin-class hardware is reported at low single-frame latencies by Orin Nano (e.g., ~16 ms by [3]) and high frame rates by Orin NX with reduced precision (e.g., ~52–65 FPS by [51]). Past Jetson generations, as well as other accelerators, exhibit much lower throughput (e.g., YOLOv4-tiny ~15 FPS by Jetson Nano in [26]), highlighting the importance of the chosen hardware and batch processing optimizations. General cross-model studies (YOLOBench [8]) also suggest that, for a given latency budget, carefully scaled detectors may perform well for mAP while satisfying real-time requirements. In addition to GPU-equipped edge boards, Mahmud et al. [52] demonstrated that a lightweight YOLOv4-based detector exported via ONNX can run in real time on a HoloLens 2 mixed-reality headset, achieving 98.76% mAP for 11 medication vial classes, with an average processing time of approximately 68 ms per frame (~15 FPS). This result complements Jetson-based benchmarks by demonstrating simi-

lar latency–accuracy trade-offs on untethered, head-mounted edge hardware. Compared to these baseline studies, the present work targets the deployment reliability and operational limits of YOLOv8 on the NVIDIA Jetson Orin NX. Prior evaluations on Jetson-class and other edge platforms [3,8,26,51,52] primarily report latency, FPS, and mAP under default or relatively small batch configurations, offering limited insight into the stability implications of aggressive batching within constrained memory budgets. Specifically, previous works have not systematically identified the batch size regimes at which GPU memory pressure leads to inference engine failures or runtime instability. In this context, our study contributes by (i) conducting a controlled evaluation of PyTorch and TensorRT implementations across five YOLOv8 variants (n/s/m/l/x) on the Orin NX platform; (ii) quantifying how latency, throughput, and GPU/RAM usage scale with batch size and input resolution while reporting explicit failure thresholds for TensorRT-based deployments; and (iii) linking these findings to two representative edge use cases—vehicle detection and package detection/counting—to derive deployment-oriented guidelines that balance throughput, accuracy, and system stability.

Table 1. Representative edge object detection benchmarks.

Work	Device(s)	Models	Key Results
Alqahtani et al. [3] (2024)	Jetson Orin; RPi 3/4/5; Edge TPU	YOLOv8, EffDet-Lite, SSD	Orin: YOLOv8n $\approx$ 16 ms; YOLOv8m $\approx$ 50 ms. COCO mAP $\approx$ 31–44.
Rey et al. [51] (2025)	Orin NX, Nano, RPi 5	YOLOv8n/s	NX: YOLOv8n $\approx$ 52 FPS (FP16), 65 FPS (INT8). UAV latency analysis.
Feng et al. [26] (2022)	Nano, Xavier, RPi4+NCS2	YOLOv3/v4 (tiny)	Nano: YOLOv4-tiny $\approx$ 15 FPS. RPi4 + NCS2: v3-tiny 7.6 $\times$ faster than v3.
Lazarevich et al. [8] (2023)	Nano; RPi 4; VIM3; x86	550+ YOLO variants	YOLOv7 variant: mAP _{50–95} = 0.657 on VOC at $\leq$ 0.1 s. Older backbones competitive with modern training.
Mahmud et al. [52] (2025)	HoloLens 2 MR headset	Lightweight YOLOv4 (Tiny-based, ONNX)	Real-time mixed-reality vial identification: mAP = 98.76% on 11 classes; $\sim$ 68 ms per frame ($\approx$ 15 FPS) with on-device inference.

In the rest of this paper, we therefore (i) reproduce a standardized benchmarking protocol on Orin NX across YOLOv8 scales, (ii) report both latency/FPS and resource footprints (GPU/RAM) under PyTorch and TensorRT, and (iii) discuss deployment implications for target real-time applications.

3. Methodology

Our methodology is structured into five phases, as shown in Figure 1. Phase I represents the environment configuration and hardware initialization on the edge device. Phase II consists of data acquisition and preprocessing, including dataset preparation and augmentation. Phase III summarizes the model configuration and training parameters. Phase IV outlines the TensorRT optimization steps. The last phase concludes the inference and the evaluation metrics used to benchmark performance.

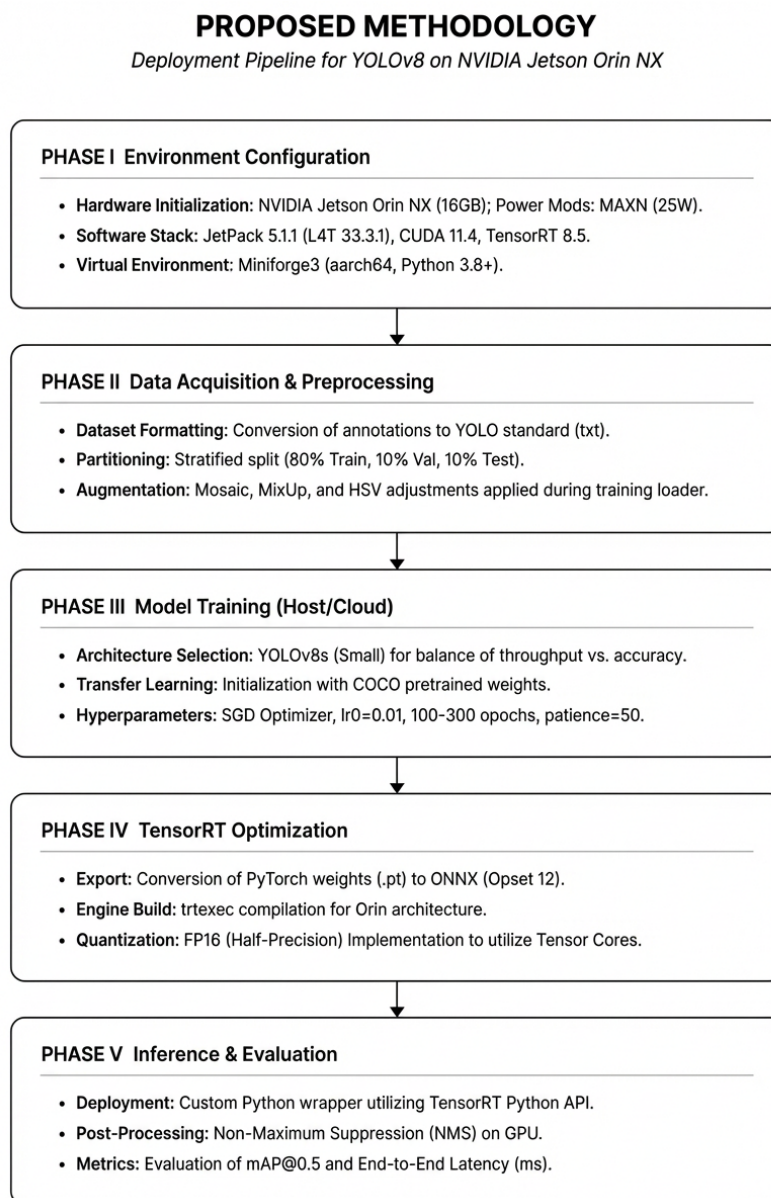


Figure 1. Proposed methodology for deployment of YOLOv8 variants for object detection efficiency on Jetson Orin NX for edge computing.

3.1. Environment Configuration and Hardware Setup

The experiments were conducted on a custom edge AI hardware platform featuring the NVIDIA Jetson Orin NX System-on-Module (SOM) with 16 GB memory. The device was initialized in MAXN (25W) power mode to ensure maximum performance. The software stack was configured using JetPack 5.1.1 (L4T 35.3.1), which provides the necessary drivers and libraries, including CUDA 11.4 and TensorRT 8.5. NVIDIA Jetson Orin NX represents the critical “mid-range” of the current edge AI market. Unlike the Orin Nano (which lacks sufficient tensor cores for heavy batching) or the AGX Orin (which is often cost-prohibitive for mass deployment), the Orin NX offers the performance-per-dollar sweet spot for commercial applications.

3.2. Data Acquisition and Preprocessing

We evaluated the models using two distinct use cases, applying a stratified split of 80% for training, 10% for validation, and 10% for testing across our datasets.

- **Vehicle Detection:** We utilized a test video corpus comprising 2644 frames capturing various traffic conditions.
- **Package Detection:** We leveraged a comprehensive dataset sourced from Roboflow, specifically curated for package and shipping label identification. This dataset contains 12,042 training images, 1187 validation images, and 16 dedicated test images [53].

Before training, annotations were converted to the standard YOLO format. To enhance model robustness, we applied data augmentation techniques during the training loader phase—for example, Mosaic, MixUp, and HSV adjustments.

3.3. Model Configuration and Training

In this study, we focus on the performance evaluation of five variants of YOLOv8, ranging from Nano to Extra Large, as shown in Table 2. These models represent a comprehensive resource spectrum from 3.0 M to 68.2 M parameters [40].

Table 2. Comparison of YOLOv8 variants.

YOLOv8 Variant	Parameters (M)	GFLOPs	Input Size
YOLOv8n (Nano)	3.0	8.2	640 × 640
YOLOv8s (Small)	11.2	28.6	640 × 640
YOLOv8m (Medium)	25.9	78.9	640 × 640
YOLOv8l (Large)	43.7	165.2	640 × 640
YOLOv8x (Extra Large)	68.2	257.8	640 × 640

Training was performed in Google Colab, which delivers higher training throughput than edge devices, while the Jetson Orin is powerful for on-device inference. We applied transfer learning initialized with COCO pretrained weights. We utilized the Stochastic Gradient Descent (SGD) optimizer with an initial learning rate (*lr*₀) of 0.01 and 300 epochs for training duration, with an early stopping patience of 50 epochs to prevent overfitting.

3.4. TensorRT Optimization

We extended beyond the native PyTorch format in order to maximize inference throughput on the Orin NX, with the following steps:

1. **Export:** The trained PyTorch weights were converted to the ONNX format.
2. **Engine Build:** We utilized the trtexec tool to compile the ONNX models into TensorRT engines specifically optimized for the Orin architecture.
3. **Quantization:** We utilized **FP16 (half-precision)** quantization to leverage the tensor cores available on the Jetson Orin.

State-of-the-art studies have shown that converting models to TensorRT engines significantly improves inference speed and reduces latency [5,27], crucial for real-time applications [54].

3.5. Inference and Performance Evaluation Metrics

For deployment, we utilized a custom Python (v3.10.12) wrapper leveraging the TensorRT Python API. Post-processing, including Non-Maximum Suppression (NMS), was executed on the GPU to minimize host–device synchronization overhead. We applied our proposed experimental workflow, as illustrated in Figure 2, to evaluate performance, with each experimental run executed ten times to ensure reliability.

Inference Time and Latency: Measured in milliseconds (ms), covering the end-to-end duration from input to output. This metric is critical for latency-sensitive applications [9].

Processing Speed (FPS): Frames per second (FPS) was measured across varying batch sizes (1, 2, 4, 8, 16, 32, 64, and 128) to determine real-time processing capabilities [10].

Resource Utilization: We monitored system-level resources, including GPU memory (initial, peak, and dynamic readings) and RAM usage, to capture baseline behavior and performance spikes [3].

Resolution Scaling: Performance was tested at both 640×640 and 1200×1200 pixels to characterize performance under different input sizes. Testing both resolutions quantified FPS, latency variance and GPU utilization, and memory consumption.

Model Format Efficiency: We compared the native PyTorch format against the TensorRT-optimized format to quantify the gains achieved through hardware-specific optimizations [7,55].

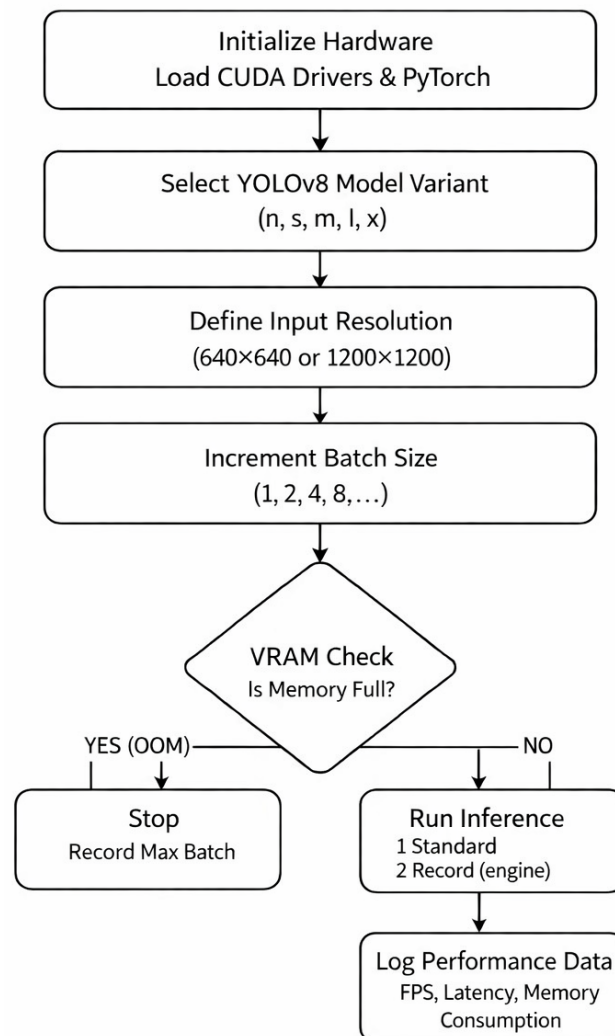


Figure 2. Experimental workflow.

4. Practical Application Use Cases

4.1. Vehicle Detection and Counting System

The first use case in this study is a vehicle detection and counting system designed for traffic management and surveillance applications. This system leverages the pretrained YOLOv8 object detection model to identify and count vehicles from video input in real time; it processes video frames to detect predefined vehicle classes such as cars, motorcycles, buses, and trucks; and it overlays annotations onto the output video at the post-processing stage. Counting is performed based on a baseline-crossing logic, where the number of detected vehicles passing a predefined line is tallied, without relying on multi-frame tracking [56]. We implemented a specific mechanism to mitigate counting errors caused

by object jitter or slow movement near the counting line. The updating counts when the bounding box of detected object transitions across predefined line. In this experiment, we first evaluated the accuracy detection for different variants of YOLOv8, as stated in Figure 3, which illustrates the trade-off of accuracy versus latency. The horizontal axis represents the base runtime, in seconds, while the vertical axis shows the accuracy metric. As we noted, there is a clear positive correlation between runtime and accuracy, with YOLOv8x achieving the highest accuracy (0.75) but requiring the longest processing time (144.636 s). YOLOv8n offers the fastest runtime (105.894 s) but with the lowest accuracy (0.61).



Figure 3. Accuracy vs. processing time analysis across YOLOv8 variants. The gray dashed line represents the performance-efficiency frontier ($R^2 = 0.97$), indicating an accuracy gain of ~ 0.0036 per additional second of runtime. This trend serves as a benchmark for evaluating model efficiency and guides selection based on application requirements.

4.1.1. Batch Size Optimization

In this experiment, we evaluated five YOLOv8 variants in PyTorch (v2.3.0) using batch sizes of 8, 16, 32, 64, and 128 to identify the batch size that maximizes performance throughput without breaking edge constraints.

Figure 4 shows the batch size optimization analysis for YOLOv8 variants; the x-axis represents the YOLOv8 model, while the y-axis is the inference speed for the video frames. Each model was tested ten times, and the average runtime was calculated to determine the most accurate estimate. The experiment demonstrates that all models show improved performance as the batch size increases to 64, after which some models begin to experience resource exhaustion, as indicated by missing data points, while YOLOv8n consistently maintains the fastest runtime across all batch sizes. The larger models (YOLOv8l and YOLOv8x) failed to complete processing with 128 batches, indicating memory limits. Figure 5 summarizes the percentage improvement in runtime efficiency gained by using the optimal batch size compared to the base runtime for each YOLOv8 variant. The vertical axis represents the percentage improvement in processing speed. As noted, all models benefited from batch processing, with improvements ranging from approximately 16% to 21%. YOLOv8m shows the highest relative improvement (21%), concluding that mid-sized models benefit the most from batch optimization.

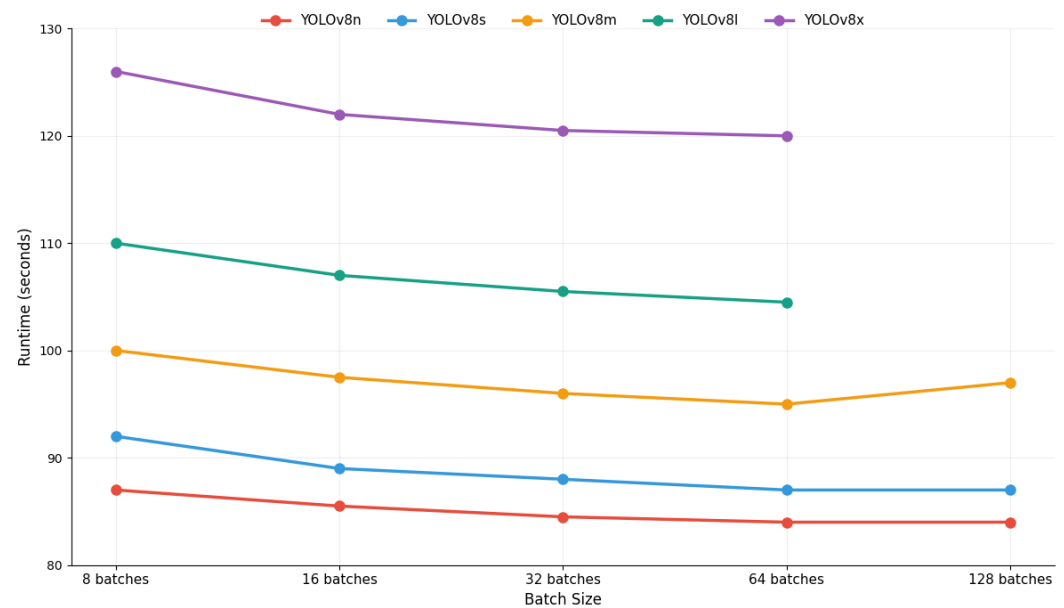


Figure 4. Batch size optimization analysis across YOLOv8 variants.

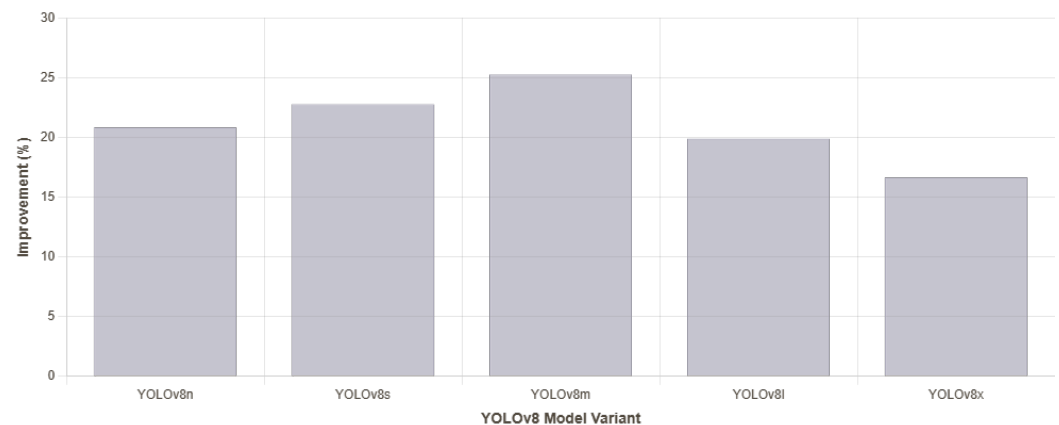


Figure 5. Percentage runtime improvement from base to optimal batch size.

We can conclude that the batching strategy generally improved the inference time, while requiring careful balancing to avoid resource exhaustion; however, it remains an effective optimization strategy regardless of model size. Meanwhile, based on our experimental evaluation on the NVIDIA Orin NX (16 GB), we observed consistent Out-of-Memory (OOM) failures when exceeding batch sizes of 64 for YOLOv8large models (YOLOv8l and YOLOv8x). Consequently, we recommend capping batch sizes at 64 to ensure system stability within the 16 GB VRAM constraint.

4.1.2. PT vs. Engine Performance

In this experiment, we evaluated the runtime performance of YOLOv8 models in two different formats—the original PyTorch (PT) format and the optimized TensorRT ENGINE format—as shown in Figure 6. The horizontal axis categorizes the five variants of the YOLOv8 model, while the vertical axis displays the runtime in seconds (lower is better). Each variant of the model has two bars that represent the two formats.

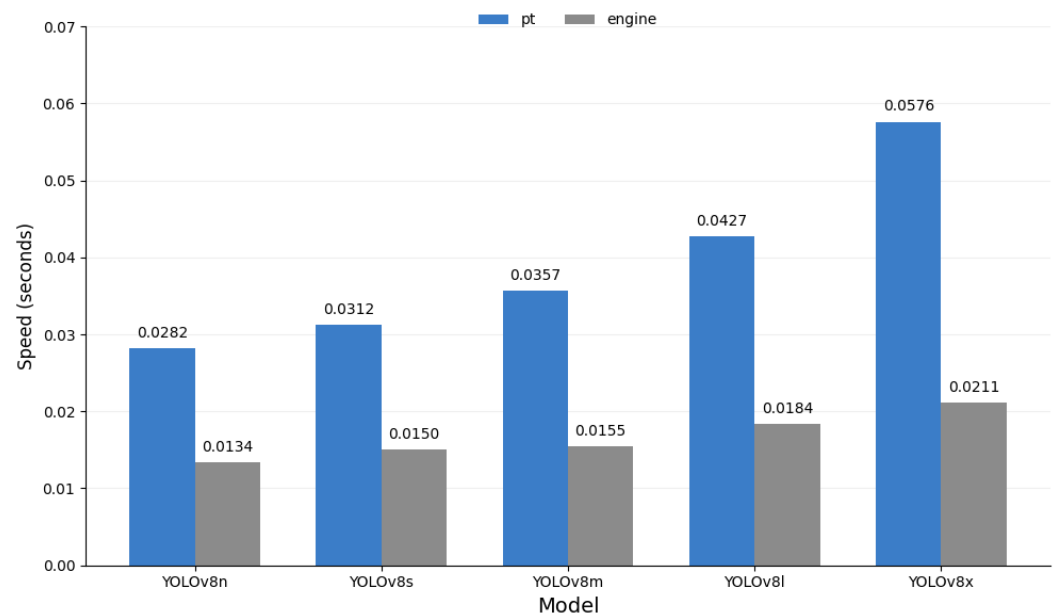


Figure 6. Optimization analysis across PyTorch vs. TensorRT (.engine).

The TensorRT ENGINE format delivered superior performance across YOLOv8 variants, with runtime reductions ranging from 52.4% to 63.3% compared to the PT format. The performance gap increases in absolute terms as model complexity increases, with YOLOv8x showing the largest absolute improvement: 36.48 s faster in ENGINE format. This demonstrates the significant impact of format optimization on inference speed, particularly for larger models, as shown in YOLOv8x, which achieved a 63.3% reduction in runtime. This demonstrates that format optimization becomes increasingly important as model complexity grows, particularly in latency-sensitive applications. In addition, this experiment effectively evaluated real-time testing using the engine format through optimized 1-batch YOLOv8 variants. The fastest result was achieved by YOLOv8n.engine, at 13.218 s.

4.1.3. Image Size Variation

In this experiment, we analyzed the speed performance of YOLOv8 variants under different resolutions (1200 and 640) with different batch sizes.

As stated in Figure 7, which focuses on high-resolution processing (1200 px), YOLOv8n with 4-batch processing (26.09 s) offers comparable performance to YOLOv8s with 4-batch processing (26.86 s), making the smaller model potentially more attractive for higher-resolution applications.

Figure 8, evaluates the input resolution with an image size of 640, and the same behavior is observed: there is a correlation between increased batch size and increased inference speed. With 4-batch processing, YOLOv8n achieves the fastest runtime, at 14.18 s, while YOLOv8l requires 27.58 s, which is 94.5% longer. This significant difference highlights the importance of model selection based on performance requirements. For real-time applications using 640 px images, batch processing with YOLOv8n or YOLOv8s provides the best balance of efficiency, with 4-batch processing reducing the runtime by 28.5% and 38.2%, respectively, compared to single-batch processing.

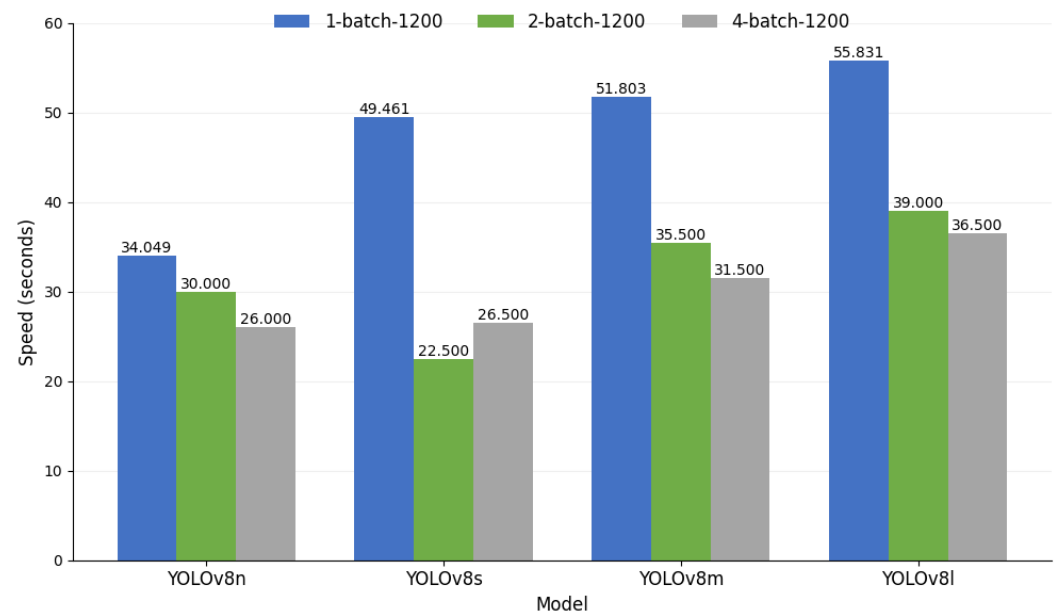


Figure 7. Batch size optimization analysis across YOLOv8 variants for image size = 1200.

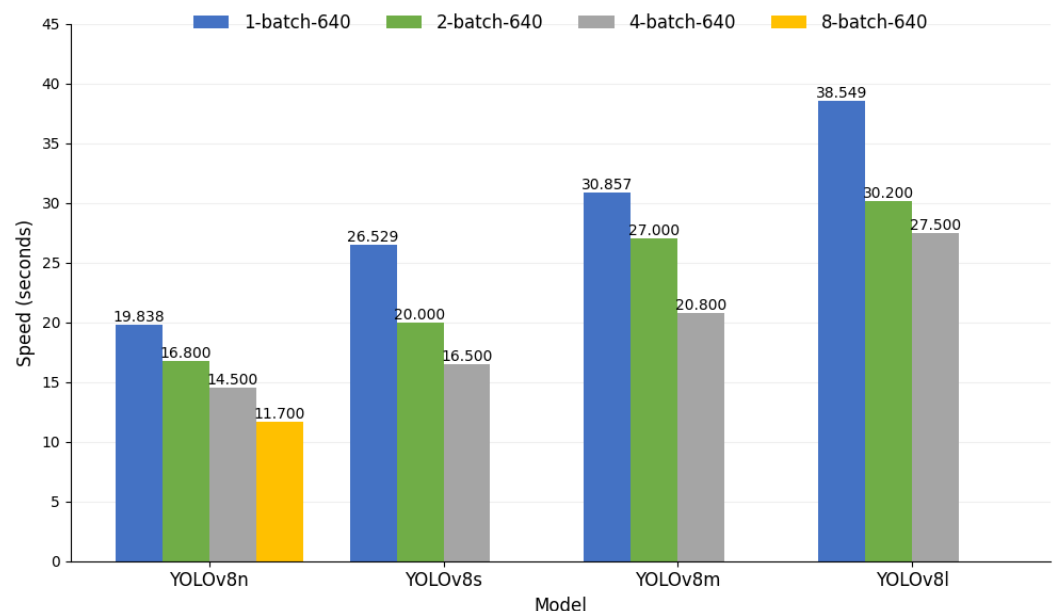


Figure 8. Batch size optimization analysis across YOLOv8 variants for image size = 640.

4.1.4. Discussion

The experiments summarize the inference speed comparison for YOLOv8 variants across different batch sizes, formats, and resolutions to determine the maximum theoretical throughput. The larger models perform faster using TensorRT compared to PyTorch, with the performance difference increasing for bigger models. Batch processing yields greater relative improvements for 640 px images compared to 1200 px images for most models. YOLOv8s shows the most significant improvement with 4-batch processing at 640 px, 38.2% faster than single-batch processing, while YOLOv8m achieves the highest improvement for 1200 px images (39.5% faster). Based on our experimental evaluation on the NVIDIA Orin NX 16 GB, we observed consistent Out-of-Memory (OOM) failures when the TensorRT Engine format exceeded batch sizes of 8. Therefore, we recommend capping batch sizes at 4 to ensure system stability within the 16 GB VRAM constraint.

4.2. Package Detection and Counting System

The second use case developed for deployment on the edge AI system involves package detection in surveillance footage. Using the YOLOv8 object detection model, the system identifies packages or parcels within the monitored area in real time. This use case demonstrates the ability of edge-based object detection to assist in logistics and security monitoring scenarios, enabling efficient visual analysis without relying on cloud processing or multi-frame object tracking.

4.2.1. Model Training

We utilized the YOLOv8n model, which was subsequently fine-tuned using the package detection dataset from Roboflow, as mentioned in Section 3.2. The training process was configured with 300 epochs with a batch size of 16, with model checkpoints saved at 50-epoch intervals. The model achieved an overall precision of 81.5% and recall of 72.2% across all classes. The ‘package’ class demonstrated superior performance, with 86.4% precision and 77.3% recall. The model achieved an mAP@0.5 of 81.7% and an mAP@0.5:0.95 of 67.3%, indicating robust performance across varying IoU thresholds. as shown in Figure 9 and Table 3. This experiment effectively leveraged Google Colab’s free GPU resources to train a specialized object detection model for package and label recognition datasets.

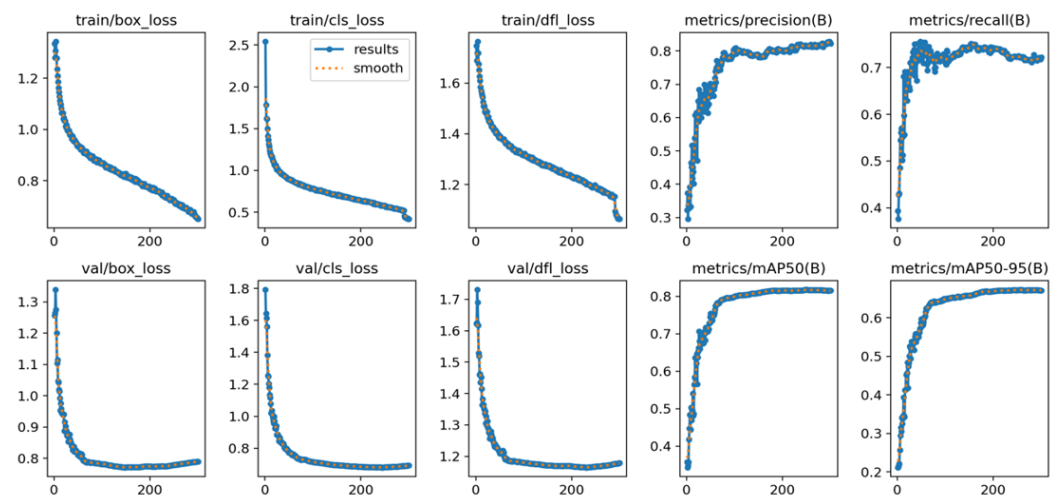


Figure 9. Training and validation performance for the YOLOv8n-based package tracking dataset over 300 epochs.

Table 3. Performance metrics of the YOLOv8 package detection model.

Metric	All Classes	Package Class
Precision	0.815	0.864
Recall	0.722	0.773
mAP@0.5	0.817	0.873
mAP@0.5:0.95	0.673	0.692

4.2.2. Inference Processing Time Comparison PT vs. Engine Performance

After training YOLOv8, we evaluated the inference speed on edge AI hardware for two types of models: TensorRT format and PyTorch format.

Figure 10 demonstrates the runtime performance comparison by batch size on TensorRT and PyTorch model formats. The TensorRT Engine format demonstrates superior performance over the PyTorch format for 1-batch, 2-batch, and 4-batch sizes, with the most significant advantage (17.7%) observed at 2 batches. However, the TensorRT Engine format crashes with 8-batch processing, while the .pt format continues to operate efficiently,

reaching its optimal performance at this batch size. This suggests that while .engine offers performance benefits for smaller batch sizes, the .pt format provides better stability and scalability for larger batch processing tasks.

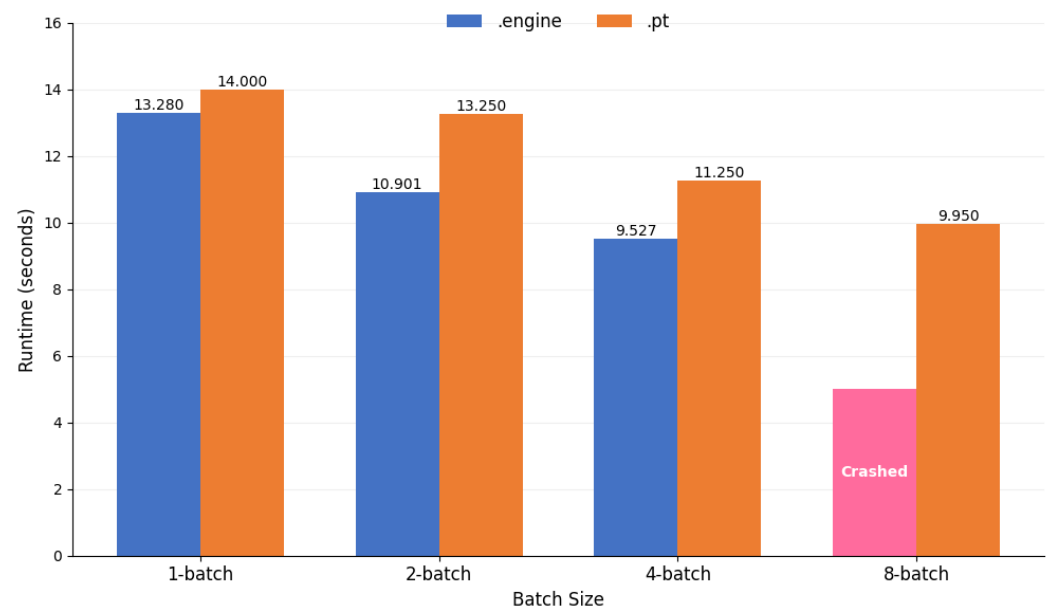


Figure 10. Inference time comparison of package detection and counting system: PyTorch vs. TensorRT engine in batches.

4.2.3. Batch Size Optimization

This section discusses the performance metrics of the package detection model, as stated in Table 4, which demonstrates notable improvements with increased batch processing. With increasing batch size from 1 batch to 4 batches, the total execution time decreased by 26.2%, from 12.72 to 9.39 s. This improvement is noted in the average frame rate, which increased from 23.67 FPS at batch size 1 to 32.04 FPS at batch size 4, demonstrating a 35.4% improvement. In addition, the batch detection time per frame decreased significantly, from 27.07 ms to 16.27 ms, which is an approximately 39.9% reduction that accounts for the majority of performance gains, while the average time per batch increases from 40.59 ms to 116.88 ms with larger batches, and the overall system performance improves due to the parallel processing advantages. These results allow us to conclude that the detection phase benefits most from batch processing, due to better GPU utilization. For real-time applications, the batch size of 4 provided the optimal balancing between processing efficiency and resource utilization on the tested edge AI hardware.

Table 4. Tracking performance by batch size.

Metric	B1	B2	B4
Batch Size	1	2	4
Total Exec. Time (s)	12.72	11.83	9.39
Avg. FPS	23.67	25.44	32.04
Seq. Batch FPS	24.63	26.61	34.22
Detect. Time (ms)	27.07 ¹	21.59 ¹	16.27 ¹
Render. Time (ms)	1.58	1.86	1.48
Time/Batch (ms)	40.59	75.17	116.88

¹ Per frame.

Increasing the batch sizes generally improved throughput but required careful balancing to avoid resource exhaustion.

4.2.4. Memory Usage Analysis

In addition to the analysis of inference time, analysis of memory usage across batch sizes, summarized in Table 5, reveals several significant patterns with important implications for model deployment. The initial memory allocation remained remarkably consistent, at around 468 MB across all batch configurations, establishing a stable baseline regardless of batch size. However, memory consumption patterns diverged significantly during processing. The 4-batch configuration consumes the most resources, with the final memory reaching 921.52 MB (2.5% higher than other configurations), and showing the largest memory increase at 453.91 MB. We noted that GPU memory usage exhibits a clear linear scaling pattern, increasing from 25.46 MB at 1 batch to 42.96 MB at 4 batches—a substantial 69% increase. Interestingly, 2-batch processing demonstrated the most efficient memory utilization, with the lowest memory increase (429.20 MB), suggesting that it may represent an optimal balance between processing throughput and resource consumption. Our findings show that larger batch sizes, while beneficial for speed, result in a marked increase in resource overhead, most notably in peak GPU memory usage. For deployment scenarios with memory constraints, the 2-batch configuration offers the most favorable efficiency profile, while environments with abundant resources may benefit from the higher throughput of 4-batch processing, despite its elevated memory requirements.

Table 5. Memory usage comparison by batch size.

Metric	Batch 1	Batch 2	Batch 4
Initial Memory (MB)	467.64	469.61	467.62
Final Memory (MB)	901.02	898.81	921.52
Memory Increase (MB)	433.39	429.20	453.91
Max GPU Memory (MB)	25.46	31.24	42.96

5. Conclusions

This study presents a comprehensive analysis and demonstrates that increasing the batch size from 1 to 4 in TensorRT-optimized vehicle and package detection systems yields substantial performance gains with minimal resource overheads. The experimental results showed a 35.4% improvement in frame rate (reaching 32.04 FPS) and a 39.9% reduction in per-frame detection time (to 16.27 ms), while only incurring a 2.3% increase in system memory and estimated scaling of GPU memory usage from 25.46 MB to 42.96 MB. The TensorRT engine format demonstrated the efficiency of using optimized memory management to benefit from practical performance improvements. In addition, the batch size of 4 in the detection and tracking phases benefited from TensorRT's parallel processing capabilities. In conclusion, this work uniquely identifies the critical stability limits and failure points of the Jetson Orin NX, providing practitioners with actionable guidelines for reliable real-world edge deployment.

6. Limitation

While our results demonstrate the efficiency of TensorRT on the Jetson Orin NX, several limitations should be noted. First, our performance metrics were derived using a limited dataset; real-world scenarios with highly variable object densities may yield different latency profiles, and the baseline-crossing logic assumes a generally unidirectional flow of traffic, typical of highway environments. Therefore, the current system does not account for vehicles that reverse direction after crossing the line (e.g., moving back and forth) but handles minor occlusions; significant partial occlusions occurring precisely at the counting line may result in missed counts or ID switching. Second, we utilized a custom active cooling kit embedded in an edge device. In custom embedded enclosures with

passive cooling, thermal throttling could occur earlier, potentially reducing the sustained FPS below the values reported here. Finally, direct comparisons with other edge AI studies are challenging due to variations in hardware configurations (e.g., specific JetPack versions, power modes, and background process overhead), which we have attempted to minimize but could not entirely eliminate.

7. Future Work

Although we conducted a systematic performance benchmarking of multiple variants of YOLOv8 on the NVIDIA Jetson Orin NX platform—focusing on object detection tasks, inference latency, and memory consumption—future work will focus on hardware-specific constraints (specifically, power consumption, energy efficiency, and thermal behavior) in real-world deployment. Furthermore, we aim to contextualize our model’s performance against broader edge AI benchmark suites to further validate its efficiency in diverse edge computing scenarios. In addition, leveraging larger and more diverse datasets would be essential to evaluate generalization accuracy in the wild. Moreover, we will investigate INT8 quantization to achieve faster inference rates and reduce computational loads on edge hardware. Applying DeepStream SDK integration to utilize the advanced memory management features will minimize latency and maximize throughput for multi-stream video analytics. In conclusion, while this work focused on YOLOv8, we observed that the linear scaling of memory with batch size and the efficiency gains from TensorRT optimization are applicable to other CNN-based architectures (such as EfficientDet or ResNet) deployed on NVIDIA hardware, as they rely on the same underlying CUDA and tensor core primitives.

Author Contributions: Conceptualization H.M.A.; methodology, H.M.A. and N.A.A.; software, A.M.A., S.O.A. and R.A.A.; validation, H.M.A., N.A.A. and M.S.A.; formal analysis, H.M.A. and N.A.A.; investigation, H.M.A. and N.A.A.; resources, S.S.A., N.U.S. and R.A.; data curation, A.M.A., R.A.A. and S.O.A.; writing—original draft preparation, H.M.A., N.A.A. and A.M.A.; writing—review and editing, H.M.A. and N.A.A.; visualization, R.A.A. and S.O.A.; supervision, M.S.A. and S.S.A. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Data sources are contained within the article.

Conflicts of Interest: Authors Najam Us Saqib, Syed Salman Alam and Reem Alsubaie were employed by the company Elm. The remaining author declares that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

References

1. Shankar, V. Edge AI: A comprehensive survey of technologies, applications, and challenges. In Proceedings of the 2024 1st International Conference on Advanced Computing and Emerging Technologies (ACET), Ghaziabad, India, 23–24 August 2024; pp. 1–6.
2. Rosende, S.B.; Ghisler, S.; Fernández-Andrés, J.; Sánchez-Soriano, J. Implementation of an edge-computing vision system on reduced-board computers embedded in UAVs for intelligent traffic management. *Drones* **2023**, *7*, 682.
3. Alqahtani, D.K.; Cheema, M.A.; Toosi, A.N. Benchmarking deep learning models for object detection on edge computing devices. In Proceedings of the 2024 International Conference on Service-Oriented Computing, Tunis, Tunisia, 3–6 December 2024; pp. 142–150.
4. Burhanuddin, M.A. Efficient hardware acceleration techniques for deep learning on edge devices: A comprehensive performance analysis. *Khwarizmia* **2023**, *2023*, 103–112. [[CrossRef](#)]

5. Sander, J.; Cohen, A.; Dasari, V.R.; Venable, B.; Jalaian, B. On accelerating edge AI: Optimizing resource-constrained environments. *arXiv* **2025**, arXiv:2501.15014. [[CrossRef](#)]
6. Shuvo, M.M.H.; Islam, S.K.; Cheng, J.; Morshed, B.I. Efficient Acceleration of Deep Learning Inference on Resource-Constrained Edge Devices: A Review. *Proc. IEEE* **2022**, *111*, 42–91. [[CrossRef](#)]
7. Verma, G.; Gupta, Y.; Malik, A.M.; Chapman, B. Performance evaluation of deep learning compilers for edge inference. In Proceedings of the 2021 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), Portland, OR, USA, 17–21 June 2021; pp. 858–865.
8. Lazarevich, I.; Grimaldi, M.; Kumar, R.; Mitra, S.; Khan, S.; Sah, S. Yolobench: Benchmarking efficient object detectors on embedded systems. In Proceedings of the 2023 IEEE/CVF International Conference on Computer Vision, Paris, France, 2–3 October 2023; pp. 1169–1178.
9. Bulut, A.; Ozdemir, F.; Bostanci, Y.S.; Soyuturk, M. Performance evaluation of recent object detection models for traffic safety applications on edge. In Proceedings of the 2023 5th International Conference on Image Processing and Machine Vision, Macau, China, 13–15 January 2023; pp. 1–6.
10. Lema, D.G.; Usamentiaga, R.; García, D.F. Quantitative comparison and performance evaluation of deep learning-based object detection models on edge computing devices. *Integration* **2024**, *95*, 102127. [[CrossRef](#)]
11. Baucas, M.J.; Spachos, P. Edge-based platform for large language models deployment in Internet of Things applications. *IEEE Netw. Lett.* **2025**, Early access. [[CrossRef](#)]
12. Lyu, Z.; Xiao, M.; Xu, J.; Skoglund, M.; Renzo, M. The larger the merrier? Efficient large AI model inference in wireless edge networks. *arXiv* **2025**, arXiv:2505.09214. [[CrossRef](#)]
13. Kania, N.; Bogliolo, A.; Calisti, L.; Contoli, C.; Fabrizio, G.D.; Romanelli, L.; Lattanzi, E. A Vision-based Virtual Sensor to Enhance Privacy and Energy Efficiency on Edge Computing. In Proceedings of the 2024 IEEE International Conference on Pervasive Computing and Communications Workshops and Other Affiliated Events (PerCom Workshops), Biarritz, France, 11–15 March 2024; pp. 625–630.
14. Logashov, D.; Shadrin, D.; Somov, A.; Pukalchik, M.; Uryasheva, A.; Gupta, H.P.; Rodichenko, N. Apple trees diseases detection through computer vision in embedded systems. In Proceedings of the 2021 IEEE 30th International Symposium on Industrial Electronics (ISIE), Kyoto, Japan, 20–23 June 2021; pp. 1–6.
15. Peng, Z.; Li, J.; Hao, H.; Zhong, Y. Smart structural health monitoring using computer vision and edge computing. *Eng. Struct.* **2024**, *319*, 118809. [[CrossRef](#)]
16. Mahendran, J.K.; Barry, D.T.; Nivedha, A.K.; Bhandarkar, S.M. Computer vision-based assistance system for the visually impaired using mobile edge artificial intelligence. In Proceedings of the 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition, Nashville, TN, USA, 19–25 June 2021; pp. 2418–2427.
17. da Silva, J.C.F.; Silva, M.C.; Luz, E.J.S.; Delabrida, S.; Oliveira, R.A.R. Using mobile edge AI to detect and map diseases in citrus orchards. *Sensors* **2023**, *23*, 2165. [[CrossRef](#)]
18. Zhang, W.; Liu, Y.; Chen, K.; Li, H.; Duan, Y.; Wu, W.; Shi, Y.; Guo, W. Lightweight fruit-detection algorithm for edge computing applications. *Front. Plant Sci.* **2021**, *12*, 740936. [[CrossRef](#)]
19. Hossain, M.E.; Tarafder, M.T.R.; Ahmed, N.; Noman, A.A.; Sarkar, M.I.; Hossain, Z. Integrating AI with edge computing and cloud services for real-time data processing and decision making. *Int. J. Multidiscip. Sci. Arts* **2023**, *2*, 252–261.
20. Seitbattalov, Z.Y.; Canbolat, H.; Moldabayeva, Z.S.; Kyzirkanov, A.E. An intelligent automatic number plate recognition system based on computer vision and edge computing. In Proceedings of the 2022 International Conference on Smart Information Systems and Technologies (SIST), Nur-Sultan, Kazakhstan, 28–30 April 2022; pp. 1–5.
21. Bouzidi, H.; Ouarnoughi, H.; Niar, S.; Cadi, A.A.E. Performance modeling of computer vision-based cnn on edge gpus. *ACM Trans. Embed. Comput. Syst. (TECS)* **2022**, *21*, 1–33.
22. Chen, Y.; Wang, C.; Lou, S. Edge artificial intelligence camera network: An efficient object detection and tracking framework. *J. Electron. Imaging* **2022**, *31*, 033030. [[CrossRef](#)]
23. Redmon, J.; Divvala, S.; Girshick, R.; Farhadi, A. You only look once: Unified, real-time object detection. In Proceedings of the 2016 IEEE Conference on Computer Vision and Pattern Recognition, Las Vegas, NV, USA, 27–30 June 2016; pp. 779–788.
24. Ma, B.; Hua, Z.; Wen, Y.; Deng, H.; Zhao, Y.; Pu, L.; Song, H. Using an improved lightweight YOLOv8 model for real-time detection of multi-stage apple fruit in complex orchard environments. *Artif. Intell. Agric.* **2024**, *11*, 70–82. [[CrossRef](#)]
25. Liang, S.; Wu, H.; Zhen, L.; Hua, Q.; Garg, S.; Kaddoum, G.; Hassan, M.M.; Yu, K. Edge YOLO: Real-time intelligent object detection system based on edge-cloud cooperation in autonomous vehicles. *IEEE Trans. Intell. Transp. Syst.* **2022**, *23*, 25345–25360. [[CrossRef](#)]
26. Feng, H.; Mu, G.; Zhong, S.; Zhang, P.; Yuan, T. Benchmark analysis of yolo performance on edge intelligence devices. *Cryptography* **2022**, *6*, 16. [[CrossRef](#)]

27. Shafi, O.; Rai, C.; Sen, R.; Ananthanarayanan, G. Demystifying TensorRT: Characterizing Neural Network Inference Engine on NVIDIA Edge Devices. In Proceedings of the 2021 IEEE International Symposium on Workload Characterization (IISWC), Storrs, CT, USA, 7–9 November 2021; pp. 226–237.
28. Jeong, E.; Kim, J.; Ha, S. TensorRT-based Framework and Optimization Methodology for Deep Learning Inference on Jetson Boards. *ACM Trans. Embed. Comput. Syst. (TECS)* **2022**, *21*, 1–26. [\[CrossRef\]](#)
29. Almurshed, O.; Patros, P.; Huang, V.; Mayo, M.; Ooi, M.; Chard, R.; Chard, K.; Rana, O.; Nagra, H.; Baughman, M.; et al. Adaptive edge-cloud environments for rural AI. In Proceedings of the 2022 IEEE International Conference on Services Computing (SCC), Barcelona, Spain, 11–15 July 2022; pp. 74–83.
30. Sivabalan, S.; Minu, R.I. Edge AI for Industrial IoT Applications. In *Applied Edge AI: Concepts, Platforms, and Industry Use Cases*; Raj, P., Nagarajan, G., Minu, R.I., Eds.; Auerbach Publications: Boca Raton, FL, USA, 2022; pp. 147–170.
31. Grzesik, P.; Mrozek, D. Combining machine learning and edge computing: Opportunities, challenges, platforms, frameworks, and use cases. *Electronics* **2024**, *13*, 640. [\[CrossRef\]](#)
32. Makina, H.; Letaifa, A.B. Bringing intelligence to Edge/Fog in Internet of Things-based healthcare applications: Machine learning/deep learning-based use cases. *Int. J. Commun. Syst.* **2023**, *36*, e5484. [\[CrossRef\]](#)
33. Mala, D.J.; Reynold, A.P. Edge Analytics with Machine Learning Technique for Medical IoT Applications. In *Advances in Computational Intelligence and Robotics*; IGI Global: Hershey, PA, USA, 2022; pp. 131–142. [\[CrossRef\]](#)
34. Polese, M.; Jana, R.; Kounev, V.; Zhang, K.; Deb, S.; Zorzi, M. Machine learning at the edge: A data-driven architecture with applications to 5G cellular networks. *IEEE Trans. Mobile Comput.* **2020**, *20*, 3367–3382. [\[CrossRef\]](#)
35. Jain, P.; Pateria, N.; Anjum, G.; Tiwari, A.; Tiwari, A. Edge AI and On-Device Machine Learning for Real Time Processing. *Int. J. Innov. Res. Comput. Commun. Eng.* **2023**, *12*, 8137–8146. [\[CrossRef\]](#)
36. Gupta, R.; Das, S.; Pal, S.K. EdgeAI Use Cases for Algorithmic Government. In *EdgeAI for Algorithmic Government*; Springer: Berlin/Heidelberg, Germany, 2023; pp. 57–65.
37. Ferreira, L.; Silva, L.; Pinho, D.; Morais, F.; Martins, C.M.; Pires, P.M.; Fidalgo, P.; Rodrigues, H.; Cortez, P.; Pilastri, A. A federated machine learning approach to detect international revenue share fraud on the 5G edge. In Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing, Virtual, 25–29 April 2022; pp. 1432–1439.
38. Arjunan, G. Optimizing Edge AI for real-time data processing in IoT devices: Challenges and solutions. *Int. J. Sci. Res. Manag. (IJSRM)* **2023**, *11*, 944–953. [\[CrossRef\]](#)
39. Pasupuleti, M.K. AI-Enabled Edge Computing: Revolutionizing IoT with Real-Time Optimization. *Int. J. Acad. Ind. Res. Innov.* **2024**, *4*, 29–64. [\[CrossRef\]](#)
40. Ultralytics. *Ultralytics YOLOv8*. GitHub Repository. 2023. Available online: <https://github.com/ultralytics/ultralytics> (accessed on 1 October 2025).
41. Ultralytics. *Ultralytics YOLOv5u: Anchor-Free Split Ultralytics Head*. *Ultralytics Documentation*. 2024. Available online: <https://docs.ultralytics.com/models/yolov5/> (accessed on 1 October 2025).
42. Li, C.; Li, L.; Jiang, H.; Weng, K.; Geng, Y.; Li, L.; Ke, Z.; Li, Q.; Cheng, M.; Nie, W.; et al. YOLOv6: A Single-Stage Object Detection Framework for Industrial Applications. *arXiv* **2022**, arXiv:2209.02976. [\[CrossRef\]](#)
43. Wang, C.-Y.; Bochkovskiy, A.; Liao, H.-Y.M. YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. *arXiv* **2022**, arXiv:2207.02696.
44. Sohan, M.; Thotakura, S.R.; Rami Reddy, C.V. A review on YOLOv8 and its advancements. In *International Conference on Data Intelligence and Cognitive Informatics*; Springer: Berlin/Heidelberg, Germany, 2024; pp. 529–545.
45. Elhanashi, A.; Dini, P.; Saponara, S.; Zheng, Q. TeleStroke: Real-time stroke detection with federated learning and YOLOv8 on edge devices. *J. Real-Time Image Process.* **2024**, *21*, 121. [\[CrossRef\]](#)
46. Koay, H.V.; Chuah, J.H.; Chow, C.-O.; Chang, Y.-L.; Yong, K.K. YOLO-RTUAV: Towards real-time vehicle detection through aerial images with low-cost edge devices. *Remote Sens.* **2021**, *13*, 4196. [\[CrossRef\]](#)
47. Ahmed, I.; Ahmad, M.; Siddiqi, M.U.R.; Chehri, A.; Jeon, G. Towards AI-Powered Edge Intelligence for Object Detection in Self-Driving Cars: Enhancing IoV Efficiency and Safety. *IEEE Internet Things J.* **2025**, *12*, 16990–16997. [\[CrossRef\]](#)
48. Li, J.; Ye, J. Edge-YOLO: Lightweight infrared object detection method deployed on edge devices. *Appl. Sci.* **2023**, *13*, 4402. [\[CrossRef\]](#)
49. Humes, E.; Navardi, M.; Mohsenin, T. Squeezed edge yolo: Onboard object detection on edge devices. *arXiv* **2023**, arXiv:2312.11716. [\[CrossRef\]](#)
50. Karim, M.J.; Nahiduzzaman, M.; Ahsan, M.; Haider, J. Development of an early detection and automatic targeting system for cotton weeds using an improved lightweight YOLOv8 architecture on an edge device. *Knowl.-Based Syst.* **2024**, *300*, 112204. [\[CrossRef\]](#)
51. Rey, L.; Bernardos, A.M.; Dobrzycki, A.D.; Carramiñana, D.; Bergesio, L.; Besada, J.A.; Casar, J.R. A performance analysis of you only look once models for deployment on constrained computational edge devices in drone applications. *arXiv* **2025**, arXiv:2502.15737. [\[CrossRef\]](#)

52. Mahmud, B.U.; Hong, G.; Lalwani, V.R.; Brown, N.; Asher, Z.D. Real-time identification of look-alike medical vials using mixed reality-enabled deep learning. *Future Internet* **2025**, *17*, 223. [CrossRef]
53. packageandbarcode. Package and Label Detection Object Detection Model (v4). Roboflow Universe. 17 September 2023. Available online: <https://universe.roboflow.com/packageandbarcode/package-and-label-detection-agjv1/dataset/4> (accessed on 14 August 2025).
54. Belhaoua, A.; Kimpe, T. TensorRT-based Surgical Instrument Detection Assessment for Deep Learning on Edge Computing. In *Medical Imaging 2024: Image-Guided Procedures, Robotic Interventions, and Modeling*; SPIE: Bellingham, WA, USA, 2024; Volume 12928, pp. 368–371.
55. P.B., G.; Marion Lincy G., R.; Rishekeeshan, A.; Deekshitha. Accelerating Native Inference Model Performance in Edge Devices using TensorRT. In Proceedings of the 2024 IEEE Recent Advances in Intelligent Computational Systems (RAICS), Kothamangalam, India, 16–18 May 2024; pp. 1–7. [CrossRef]
56. VuBacktracking. Vehicle Tracking and Counting Using YOLO and ByteTrack. GitHub Repository. 2025. Available online: <https://github.com/VuBacktracking/yolo-bytetrack-vehicle-tracking?tab=readme-ov-file#usage> (accessed on 1 October 2025).

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.